

LangChain Open Deep Research

架构演化、源码机制、Context Engineering 与工程实践完整研究

研究重点：know-why × know-how

Scope / Supervisor / Researcher / Compression / MCP / Model Routing / LangGraph State / Evaluation /
Production Lessons

核心判断：Open Deep Research 最值得学习的并不是某个 Prompt，而是它如何随着模型能力变化不断“移除结构”：从固定章节 workflow，演化到动态 supervisor、多研究单元的上下文隔离，再到“并行只用于信息收集、写作统一在最后一步完成”。

版本/状态基线：GitHub main，检索日期 2026-08-13

截至检索时：12,605 Stars / 1,784 Forks；仓库最近 push 2026-08-10；MIT License。

目录与阅读建议

1. 执行摘要：它解决什么问题，为什么值得学
 2. 项目现状与证据边界
 3. 架构演化：Workflow → Multi-Agent → 当前 Agentic Research
 4. 当前主架构：Scope → Research → Write
 5. Phase 1 Scope：Clarification 与 Research Brief
 6. Phase 2 Research：Supervisor 的动态研究控制
 7. Researcher Subgraph：独立 ReAct / Tool Loop
 8. Context Engineering：压缩发生在哪里、为什么有效
 9. Search / Web / MCP Tool Plane
 10. Model Routing：四类模型如何分工
 11. LangGraph State / Reducer / Subgraph 机制
 12. 并发、预算、终止条件与成本模型
 13. Citation / Provenance 设计
 14. Final Report：为什么选择 one-shot 统一写作
 15. 异常、token limit、retry 与生产级韧性
 16. Evaluation：DeepResearch Bench、结果与解释边界
 17. 2026 当前风险与开放问题
 18. Legacy 实现对照：旧结构为什么被保留为教材
 19. 与 GPT Researcher / DeerFlow 的工程定位对比
 20. 可沉淀的 Know-Why
 21. 可直接复用的 Know-How
 22. 从零实现一个 Open Deep Research MVP
 23. 改进建议与下一代架构
- 附录 A. 核心源码阅读地图
- 附录 B. 主要资料与引用

推荐阅读顺序：如果目的是学习架构，优先看 3→4→6→7→8→20；如果目的是自己实现，优先看 4→9→10→11→12→22；如果目的是做评测和生产，优先看 15→16→17→23。

1. 执行摘要：Open Deep Research 真正解决了什么

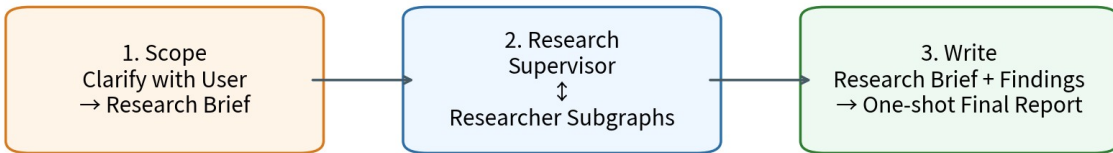
Deep Research 不是“搜索 API + LLM 总结”的放大版。它的难点在于：用户请求通常是开放式的，所需搜索深度、并行度、证据数量、来源类型和最终组织方式无法在任务开始前完全确定。LangChain 官方博客把这一点直接定义为核心挑战：研究策略必须根据请求类型和中间结果动态变化，而不是预先固定。

一句话模型：Open Deep Research = 一个会先定义成功标准（Research Brief）、再动态调度独立研究单元、在边界处压缩上下文、最后统一综合写作的 LangGraph 研究控制系统。

- Scope：把“用户聊天历史”压缩为明确 research brief，作为后续所有判断的 north star。
- Research：Supervisor 不直接搜网页，而是决定“是否需要拆分、拆成什么、并行多少、还缺什么”。
- Researcher：每个研究单元只负责一个 topic，在隔离 context window 里运行 tool-calling loop。
- Compression：Researcher 结束后不把 raw web/tool logs 直接交给 Supervisor，而是先清理、去重、保留引用。
- Write：只在研究完成后进行一次统一写作，避免多 agent 并行写章节导致报告割裂。
- Configuration：Search、MCP、研究模型、摘要模型、压缩模型、最终写作模型以及并发/迭代预算均可配置。

从源码看，当前图的顶层节点只有四个：`clarify\_with\_user`、`write\_research\_brief`、`research\_supervisor`、`final\_report\_generation`。复杂性主要被下沉到 supervisor subgraph 和 researcher subgraph，而不是堆在顶层 workflow。这个设计是理解 LangGraph “低层图 + 可组合 subgraph” 价值的一个非常好的样本。

当前主流程：Scope → Research → Write



关键边界：Scope 压缩“用户上下文” | Research 压缩“工具上下文” | Write 只消费清洗后的研究结果

这不是一个固定“报告章节流水线”，而是一个让 Supervisor 动态决定研究深度与并行度的研究控制系统。

2. 项目现状与证据边界

维度	当前状态（2026-08-13）	研究意义
仓库	langchain-ai/ open_deep_research	LangChain 官方开源实现
Stars / Forks	12,605 / 1,784	生态影响力较高，但不等价于质量
最近 push	2026-08-10	项目仍活跃
主语言	Python	核心 runtime 简洁，源码集中
License	MIT	适合二次开发
主图入口	deep_researcher.py:deep_researcher	LangGraph Server 直接加载该 graph
官方评测	DeepResearch Bench	README 保留 2025 结果；基准评估器在 2026 已迁移

证据采用三层：第一层是 2026-08-13 的 GitHub API / main 源码；第二层是 LangChain 2025 官方架构博客；第三层是 DeepResearch Bench 官方仓库。报告中的“工程判断”会明确作为分析，不与官方事实混写。

重要时效性说明：仓库 README 里记录的 DeepResearch Bench 分数来自 2025 的评估配置；DeepResearch Bench 在 2026-05 已将官方 RACE evaluator 迁移到 GPT-5.5。因此旧分数适合做“项目当时迭代依据”，不应直接当作 2026 新榜单上的可比排名。

3. 架构演化：为什么从 Workflow 走向更少结构的 Agent

Open Deep Research 架构演化：不断移除不再必要的结构



3.1 第一阶段：Orchestrator-Worker Workflow

早期实现把研究问题先拆成固定“报告章节”，然后并行研究、并行写作。这在 tool calling 不稳定、context window 较小的时期是一个合理的工程捷径：确定性高、并发简单、失败面少。

但这里隐藏了三个强假设：① 所有研究问题都适合先拆成章节；② 章节可以彼此独立研究；③ 章节可以彼此独立写作。随着研究问题变复杂，这些假设会制造上限。

3.2 第二阶段：Multi-Agent，但仍保留“并行写章节”

随着 tool calling 与 MCP 生态成熟，LangChain 团队把研究环节改成 supervisor + sub-agents，让系统可以动态决定搜索策略。然而，如果每个 sub-agent 仍负责写自己的一章，最终报告仍可能产生观点重复、术语不一致、证据权重不一致、跨章节推理断裂等问题。

3.3 当前阶段：只并行“可独立的信息收集”，统一写作

当前架构把 multi-agent 限定在 Research phase。研究单元可以并发，但最终报告由一个 writer 基于 research brief + 全部 compressed findings 一次性生成。官方将这一变化总结为：多 agent 适合容易并行的任务，而写作属于需要全局一致性的综合任务。

Bitter Lesson 的工程含义：不要把“模型暂时做不好”的限制永久编码成架构。应当把结构做成可移除、可替换的低层模块，并随着模型能力提升重新评估。

4. 当前主架构：Scope → Research → Write

顶层 graph 的设计非常克制。它不显式建“搜索节点、抓取节点、引用节点、章节节点”，而是只把真正稳定的生命周期边界留在图上。

Phase	核心问题	主要输出	为什么要独立成阶段
Scope	用户到底要什么？	Research Brief	先定义成功标准，降低后续漂移
Research	需要搜什么、搜多深、是否并行？	Clean Findings / Notes	让策略随中间结果动态变化
Write	怎样统一组织证据并回答 brief？	Final Report	需要全局一致性，不适合分片并行写作

4.1 顶层 LangGraph

1. ``START` → clarify_with_user``
: 判断信息是否足够。2. ``clarify_with_user` → write_research_brief``
: 若无需追问，生成结构化研究简报。3. ``write_research_brief` → research_supervisor``
: 进入 supervisor subgraph。4. ``research_supervisor` → final_report_generation``
: 研究完成后统一生成报告。5. ``final_report_generation` → END``
: 清理 notes 并返回最终消息。

这里的 know-how 是：顶层 graph 不应该等同于“所有动作的流程图”。图上只放跨阶段、可观察、需要 checkpoint/恢复的生命周期节点；阶段内部用 subgraph 或 tool loop 保持灵活。

5. Phase 1 Scope：Clarification 与 Research Brief

5.1 Clarification 不是礼貌步骤，而是减少下游搜索浪费

``clarify_with_user`` 使用 research model 的 structured output ``ClarifyWithUser``，输出 ``need_clarification / question / verification``。当配置关闭 ``allow_clarification`` 时可直接跳过。

它解决的不是“让对话更自然”，而是一个资源优化问题：研究任务后续可能消耗几十次工具调用和大量 token，若 scope 错了，后面所有计算都会放大错误。

5.2 Research Brief 是一个“可执行成功标准”

``write_research_brief`` 会把完整聊天历史转换为单个结构化 ``research_brief``，并把它写入 ``AgentState``。随后 supervisor 的 system prompt 和 final writer 都会以它为主约束。

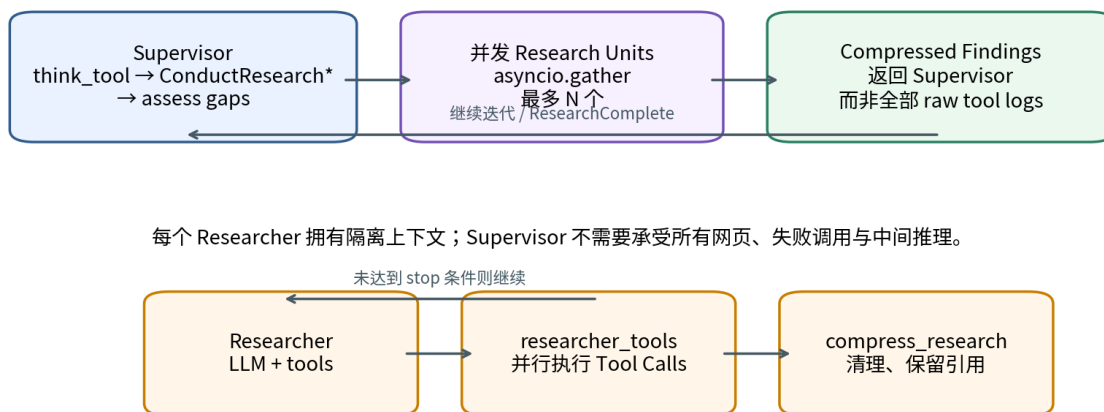
Know-why: Research Brief 的本质不是摘要，而是 Context Contract：把用户意图从“聊天语境”转成

后续 Agent 可以反复对照的任务规格。

- 减少历史聊天 token 长期驻留。
- 统一 supervisor 与 final writer 的目标函数。
- 允许用户提供样例、偏好、来源约束，但不必把所有原始对话带进每个研究单元。
- 使 eval 更清楚：最终报告是否满足 brief，而不是是否“看起来很长”。

6. Phase 2 Research：Supervisor 的动态研究控制

双层 Agent Loop：Supervisor Loop + Researcher ReAct Loop



每个 Researcher 拥有隔离上下文；Supervisor 不需要承受所有网页、失败调用与中间推理。

6.1 Supervisor 不负责“搜”，而负责“研究管理”

Supervisor 的工具只有三个概念：`ConductResearch`（委派一个详细 topic）、`ResearchComplete`（结束研究）、`think\_tool`（显式反思）。这意味着“搜索 API 的具体形态”被封装在 Researcher 内，Supervisor 只看到研究任务和压缩后的研究结果。

6.2 动态并行，而不是固定 fan-out

Supervisor prompt 明确“偏向 single agent，只有明显可并行时才拆分”。比较 A/B/C 这类天然独立维度适合多个研究单元；简单事实、单一列表等请求通常不值得启动多个 sub-agent。

源码中 `max\_concurrent\_research\_units` 默认 5。若一次模型调用生成超过上限的 `ConductResearch` tool calls，系统只执行前 N 个，并为溢出的调用返回错误信息。真正执行时使用 `asyncio.gather` 并行启动 researcher subgraph。

6.3 Supervisor Iteration 是“研究深度控制器”

每轮 supervisor 都会查看 compressed findings，再决定继续补充、换方向、还是 `ResearchComplete`。默认 `max\_researcher\_iterations=6`，是对“无限研究”的硬预算。

核心抽象：Deep Search 的深度不是预先写成 depth=3，而是由 Supervisor 基于“当前证据覆盖 vs Research Brief”动态决定。

7. Researcher Subgraph：独立 ReAct / Tool Loop

7.1 每个 Researcher 只看一个 research_topic

`ConductResearch.research\_topic` 在 state schema 中要求“单一 topic，至少一段的高细节描述”。Sub-agent 看不到其他 agent 的工作，这反而是刻意的 context isolation：每个线程只携带与自己问题相关的 tool feedback。

7.2 Loop 结构

6. Researcher 获取 `get\_all\_tools(config)`。
7. Research model 绑定 search / MCP / think_tool / ResearchComplete。
8. LLM 产生一组 tool calls。
9. `researcher\_tools` 并行执行同一轮所有 tool calls。
10. 若未满足 stop 条件，将 ToolMessage 追加回 researcher messages，继续 loop。
11. 达到 `max\_react\_tool\_calls` 或 ResearchComplete 后进入 `compress\_research`。

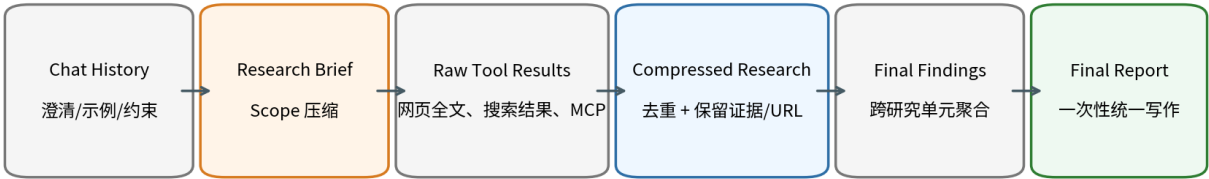
7.3 think_tool 的真正作用

官方 prompt 强制 search 后单独调用 `think\_tool`，并明确不要与 search 并行。它不是为了“展示思考过程”，而是人为插入一个 decision checkpoint：总结刚得到的证据、识别缺口、决定下一次查询是否值得。

Know-how：在高成本工具循环中，“显式反思节点”可以看作一个低成本 policy regularizer：让模型避免连续发散搜索，也给 stop condition 更清晰的决策上下文。

8. Context Engineering：压缩发生在哪里、为什么有效

Context Engineering：不是“更多上下文”，而是“正确边界上的压缩”



压缩点 A：用户对话 → focused brief，避免把聊天历史长期带入研究。

压缩点 B：每个 sub-agent 的 raw research → compressed findings，减少 token bloat / context clash。

统一写作：不让多个 writer 在隔离上下文中各写一段，从根源上减少“章节拼接感”。

8.1 两个最重要的压缩边界

压缩边界	输入	输出	主要收益
用户上下文压缩	完整 messages / 澄清 / 样例	research_brief	减少 scope 噪声；统一成功标准
研究上下文压缩	AI messages + tool outputs + 网页/MCP 数据	compressed_research	避免 Supervisor context bloating；保留证据与来源

8.2 为什么不直接把 raw tool results 返回 Supervisor

网页全文、失败工具调用、搜索结果重复、无关页面都会占据大量上下文。若 Supervisor 同时管理多个 sub-topic，把这些 raw results 合并到一个窗口会引发 token bloat、context clash、注意力稀释和 rate-limit 压力。

因此 `compress\_research` 使用独立 compression model 做清洗。值得注意的是，prompt 特别强调“不要总结到丢信息”，而是删除明显无关/重复内容、保留所有相关来源与引用。换句话说，它更接近 evidence normalization，而不是传统 summary。

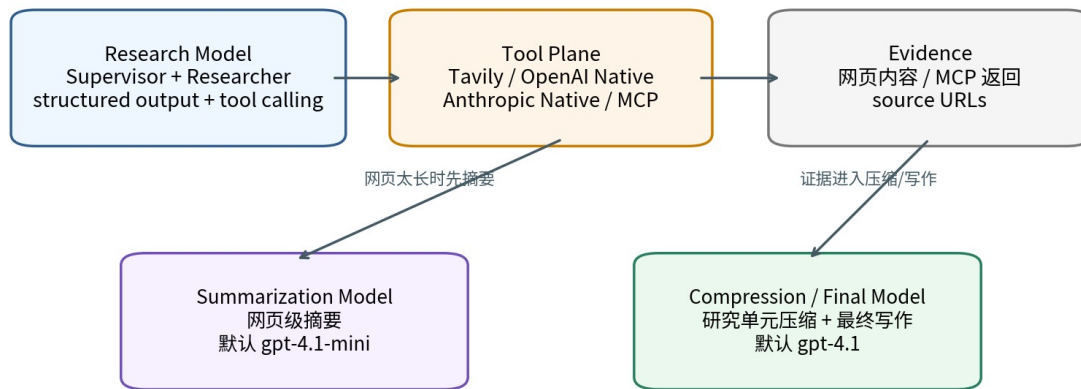
8.3 Compression 失败时的 token 兜底

`compress\_research` 最多尝试 3 次；若判断为 token-limit，会调用

`remove\_up\_to\_last\_ai\_message` 删除更老的消息后重试。这个策略表明：长任务的 context management 必须是 runtime concern，而不能只靠 prompt 要求“简洁”。

9. Search / Web / MCP Tool Plane

工具与模型分工：Research Policy 与 Data Plane 解耦



Know-how：用便宜模型处理“高频、局部、结构化”任务；把强模型预算留给研究策略、跨证据综合与最终报告。

9.1 当前 main 分支的 SearchAPI

SearchAPI	实现方式	特点
Tavily	自定义 `tavily_search` tool	支持多 query 并发、raw content、URL 去重、网页摘要
OpenAI	native `web_search_preview`	由模型原生 web search 机制执行
Anthropic	native `web_search_20250305`	配置 max_uses=5
None	无 search tool	可仅依赖 MCP 工具

注意：截至本报告源码基线，`Configuration.SearchAPI` 只列出上述选项；GitHub 上 2026 仍有 Exa、Firecrawl、SearXNG 等扩展 PR。不能把“开放 PR”误写为 main 已正式支持。

9.2 Tavily 路径值得学习的细节

12. 一次 tool call 可携带多个 `queries`，底层并发执行。

13. 结果按 URL 去重，减少相同来源重复进入上下文。

14. 请求 raw content；若网页有正文，则用独立 summarization model 做结构化 `Summary(summary, key\_excerpts)`。
15. 网页内容可由 `max\_content\_length` 截断，默认 50,000 characters。
16. 网页摘要任务并行执行；超时 60 秒则退回原始网页内容，而不是直接失败。

9.3 MCP：把“研究策略”与“企业数据源”解耦

`get\_all\_tools` 会把 search tools 与 MCP tools 合并。MCP 支持 tool allow-list、名称冲突检查、可选认证与 OAuth token exchange。研究 sub-agent 因此可以不改核心 loop，就访问内部系统、数据库或业务工具。

工程意义：Research Agent 的“智能”不应绑定某个搜索供应商。更稳定的抽象是：Supervisor 决定 research topic；Researcher 面对一个动态 Tool Plane。

10. Model Routing：为什么不是一个模型包打天下

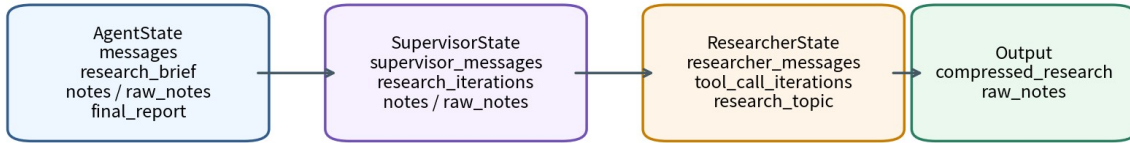
角色	默认模型（main）	调用频率/负载	为什么分开
Summarization	openai:gpt-4.1-mini	网页级，高频	局部结构化任务，可用更便宜模型
Research	openai:gpt-4.1	Supervisor + Researcher 循环	需要 tool calling、策略判断、structured output
Compression	openai:gpt-4.1	每个 research unit 结束时	需要高保真清洗、保留引用
Final Report	openai:gpt-4.1	每次任务一次	需要跨证据综合、结构与语言一致性

这是一个典型的“按认知负载路由模型”设计，而不是按组件名称路由。频繁但局部的网页摘要可以便宜；决定下一步研究方向、压缩研究证据和最终综合则需要更强模型。

源码使用 `init\_chat\_model(configurable\_fields=("model","max\_tokens","api\_key"))`，各阶段通过 runnable config 动态选择模型。这样可以在 LangGraph Studio / OAP 中把模型选择暴露为配置，而不改 graph。

11. LangGraph State / Reducer / Subgraph 机制

LangGraph State：让长任务的控制流、数据流、终止条件显式化



Reducer 设计：messages 通常追加；某些字段支持 override，避免多次迭代把旧数据无限累积。

显式状态的价值：可观测、可 checkpoint、可重放、可设置预算与错误恢复；
这也是 LangGraph 相比“一个 while tool_use 循环”的主要工程增量。

11.1 为什么需要三层 State

`AgentState`、`SupervisorState`、`ResearcherState` 分别对应生命周期、管理层和执行层。它们避免把所有内部消息都提升到主图 state，从而控制状态体积和暴露边界。

11.2 override_reducer

某些 state 字段默认是“追加”，但当 update 使用 `{type: "override", value: ...}` 时允许覆盖。例如初始化 supervisor messages、最终清空 notes。这个技巧很小，但对于长任务非常关键：否则每次 run 会不断积累旧状态。

11.3 Subgraph 是“上下文与故障域”的边界

Researcher 被编译成独立 subgraph，并被 Supervisor 并行 `ainvoke`。这使每个研究单元拥有自己的 state、tool loop 和 compression 过程。相比在一个超大主图里为每个 topic 建动态节点，subgraph 更适合同长任务的可组合性。

12. 并发、预算、终止条件与成本模型

12.1 三层预算

层级	参数	默认	控制什么
Supervisor 并发	max_concurrent_research_units	5	一轮最多并行多少 research units

层级	参数	默认	控制什么
Supervisor 深度	max_researcher_iterations	6	最多多少轮委派/反思
Researcher tool loop	max_react_tool_calls	10	单个研究单元最多多少轮工具迭代

Prompt 里还包含更保守的启发式，例如简单 query 2-3 次搜索、复杂 query 最多约 5 次搜索、连续两次搜索返回类似信息时停止。硬预算和软启发式共同抑制“研究成瘾”。

12.2 粗略成本模型

可把一次任务成本粗略理解为： $Cost \approx Scope + \sum (ResearchUnit_i \times ToolCalls_i + Compression_i) + FinalWrite$ 。

因此最有效的成本优化通常不是“把所有模型换便宜”，而是减少不必要的 research units、减少重复 search、让压缩在正确边界发生，并让 cheap summarizer 处理高频网页内容。

12.3 动态并行为什么优于固定 breadth

固定 breadth=5 会让简单问题也启动 5 个研究线程；固定 depth=3 会让已经证据充分的任务继续深挖。Supervisor-based depth control 将计算分配与当前 uncertainty / coverage 绑定，更适合真实研究请求。

13. Citation / Provenance 设计

Open Deep Research 的 citation 不是最终 writer 临时“编”的，而是在 Researcher compression 阶段就要求保留每个来源、URL 与对应信息。随后 final writer 再使用 Markdown `[Title](URL)` 形式组织引用。

- 搜索工具输出必须带 URL。
- Compression prompt 要求给每个 unique URL 分配稳定 citation number，并在末尾列 Sources。
- Compression 强调不要丢掉任何 research sources，因为后续 LLM 还要做跨研究单元合并。
- Final report prompt 要求引用相关来源并给 Sources section。

Know-why: Citation quality 的前提不是“最后一步加链接”，而是 provenance 从 retrieval → compression → synthesis 全链路不丢失。

13.1 仍然存在的风险

即使 URL 全程保留，最终 writer 仍可能发生“引用位置与具体 claim 不完全对应”或“多个 compressed reports 重新编号后冲突”。2026 的 issue 也出现过 compression citation / Sources 格式一致性问题。因此生产系统最好增加 claim-citation verifier，而不是只依赖 prompt。

14. Final Report：为什么选择 one-shot 统一写作

`final\_report\_generation` 只消费 `research\_brief + messages + findings`。它不再调用搜索工具，也不再启动 sub-agent。这个边界非常重要：研究阶段负责 coverage，写作阶段负责 coherence。

并行写章节	统一 one-shot 写作
快，但章节之间缺乏共享上下文	需要更长一次调用，但拥有全局上下文
容易术语、结论、证据权重不一致	可以做跨主题比较与去重
每个 writer 可能重复 intro/conclusion	结构可按最终 brief 动态决定
适合高度独立内容块	更适合需要整体论证的研究报告

官方博客把这一改动视为关键经验：multi-agent 只用于“可以自然并行的信息收集”，不要把需要高协调的一致性写作也拆散。

15. 异常、Token Limit、Retry 与生产级韧性

15.1 Structured Output Retry

Clarification、Research Brief、网页摘要等 structured output 默认最多重试 3 次。原因是模型即使支持 structured output，也可能出现 provider 兼容、临时格式错误或边界输出。

15.2 Tool 执行隔离

Researcher 的每个 tool call 由 `execute\_tool\_safely` 包裹，异常会转成字符串反馈给模型，而不是直接终止整个 researcher。这样 Agent 可以根据失败结果调整策略。

15.3 Compression Token Limit

如果 compression 触发 token limit，runtime 会删除较老的消息并重试。这是典型的“bounded degradation”：宁可牺牲部分旧 raw context，也要尽量得到可返回给 Supervisor 的 clean findings。

15.4 Final Report Token Limit

最终写作最多重试 3 次。第一次根据模型 context limit 估计 findings 的字符上限，后续每次再缩短约 10%。这说明长报告系统必须对“最后一步上下文爆炸”有专门兜底。

值得警惕的源码点：当前 `supervisor\_tools` 的异常分支中存在 `if is\_token\_limit\_exceeded(...) or True:`，意味着 ConductResearch 并发块中的任何异常都会直接结束该 research phase。它体现了“fail fast”倾向，但也可能过度吞掉可恢复错误；生产化应按错误类型分类处理。

16. Evaluation：DeepResearch Bench、结果与解释边界

仓库内置 `tests/run\_evaluate.py` 与结果导出脚本，可在 LangSmith dataset 上运行 DeepResearch Bench。README 给出的历史结果包括：默认 gpt-4.1 路线 RACE 0.4309、Claude Sonnet 4 0.4401、GPT-5 0.4943。

配置	README 记录的 RACE	说明
Default: gpt-4.1-mini + gpt-4.1 + gpt-4.1	0.4309	约 58M tokens；记录成本 \$45.98
Claude Sonnet 4 Research	0.4401	研究模型不同，成本显著更高
GPT-5 Research	0.4943	2025-08 更新，README 中最高

但必须区分“项目历史评测结果”和“2026 当前官方榜单”。DeepResearch Bench 官方在 2026-05 将 RACE evaluator 从 Gemini-2.5-Pro 迁移到 GPT-5.5，并计划只维护新 evaluator 的榜单。跨 evaluator 的分数不能直接比较。

16.1 为什么评测仍然有价值

- 它迫使架构优化围绕最终 report quality，而不是单次搜索准确率。
- 可以用相同 100 个任务比较不同 research model、compression model、并发策略和 prompt。
- LangSmith trace 能把最终分数与具体 supervisor/tool trajectory 对齐，适合做 agent eval。

16.2 应新增哪些 eval

- Coverage：brief 中每个明确要求是否被证据覆盖。
- Citation entailment：每个 claim 的引用页面是否真正支持该 claim。
- Research efficiency：每个有效事实对应多少 search/tool tokens。
- Redundancy：重复来源、重复 query、相似 sub-agent 的比例。
- Stop quality：系统是否在证据已经充分时及时停止。
- Robustness：单一 search/MCP provider 失败时能否降级继续。

17. 2026 当前风险与开放问题

从 2026 的 Issues / PR 可以看到，项目已从“证明架构可行”进入“修边界与扩工具生态”阶段。值得关注的问题包括 token-limit 截断方向、LangGraph API 演进、custom search provider、报告模板定制、citation rule 一致性、并发研究可见性等。

风险	当前表现	建议
Search provider 扩展	main 枚举仍较少，生态通过 PR 扩展	把 search adapter 变成插件 registry
错误传播	Supervisor 对部分异常偏 fail-fast	按 transient / auth / token / provider 分类重试
引用验证	主要靠 prompt 保留 provenance	加入 claim-citation verifier
最终写作上下文过大	通过截断 findings 兜底	引入分层 evidence store / retrieval-on-write
长期记忆	官方博客仍列为开放方向	把高成本研究结果存成可检索 research artifact
评测热路径	尚无系统化 online quality gate	对 coverage/contradiction/source quality 做轻量 verifier

18. Legacy 实现对照：旧结构为什么仍然值得读

实现	主要结构	优点	主要瓶颈
legacy/graph.py	Plan-and-Execute + HITL + 顺序章节	可控、易解释、适合教学	研究策略固定、慢、章节结构先验强
legacy/multi_agent.py	Supervisor + 多 Researcher 并行	速度快、MCP 友好	若子 agent 同时写章节，整体 coherence 差
current	Scope + Supervisor Research + Compression + One-shot Write	灵活、上下文隔离、统一写作	依赖更强的 tool calling / context engineering

Legacy 代码的价值是帮助理解“为什么当前实现长这样”。把它们当成架构消融实验，比只读 current code 更能形成 know-why。

19. 与 GPT Researcher / DeerFlow 的工程定位对比

下面是架构层面的对照，用于定位，而不是简单排名。

维度	Open Deep Research	GPT Researcher	DeerFlow 2.x
核心定位	可配置 Deep Research Agent reference	成熟研究 pipeline / research engine	Long-horizon SuperAgent Harness
研究控制	Supervisor 动态委派 subagents	经典 pipeline + recursive deep mode + multi-agent variants	Lead Agent + middleware + subagents + skills
Context 策略	Brief + subagent isolation + compression	Embedding/context compressor + research synthesis	Summarization middleware + filesystem/sandbox + skills
Tool 层	Tavily / native web / MCP	多 retriever/scrapper + MCP	Sandbox/MCP/ community tools/skills
写作策略	研究后 one-shot final write	多 report type / generator	Artifact/skill 驱动，研究只是能力之一
最适合学习	Agentic research architecture	Research pipeline 工程细节	通用 Agent runtime / harness

如何选择：如果你要理解“研究策略怎样动态化”，优先 Open Deep Research；如果你要做可直接嵌入产品的研究引擎，GPT Researcher 更丰富；如果你最终目标是一个能研究、编码、操作文件的通用长任务 Agent，DeerFlow 更接近 Harness。

20. 可沉淀的 Know-Why

- KW-1 先定义 Research Contract** — 复杂研究的首要风险不是“搜不到”，而是“搜错问题”。Research Brief 应先于大规模工具调用。
- KW-2 多 Agent 的主要价值是 Context Isolation** — 并行只是副产品；真正收益是让互不相关的 tool feedback 不在同一个上下文竞争。
- KW-3 不要让并行边界穿过强协调任务** — 研究可以独立，统一论证通常不可以。把 writer 留在最后。
- KW-4 Compression 是 Runtime Primitive** — 长任务里压缩不是“省 token 小技巧”，而是维持推理质量与可运行性的基础设施。
- KW-5 搜索深度应该由 Coverage 驱动** — 固定 depth/breadth 是简单近似；更好的 policy 是根据 brief 覆盖率和证据缺口动态继续。

KW-6 Tool Plane 要与 Research Policy 解耦 — 今天是 Tavily，明天可能是 native search、MCP、内部数据库；Agent loop 不应因此重写。

KW-7 架构要为模型变强留退路 — 结构是补丁，不是信仰。每一层手工 workflow 都应该问：未来模型变强后能否删掉？

KW-8 Eval 必须围绕最终研究价值 — 研究 agent 的优化目标不是 query 数量，而是 coverage、证据质量、citation correctness 与最终报告质量。

21. 可直接复用的 Know-How

17. 把 Clarification 做成 structured output，而不是自由文本对话逻辑。
18. 把用户对话压成 Research Brief，并作为 Supervisor 与 Writer 的共同输入。
19. Supervisor 只暴露 ConductResearch / ResearchComplete / Think 这类高层工具，不直接暴露所有搜索工具。
20. 对每个 research unit 使用独立 state/subgraph，避免跨 topic tool logs 污染。
21. 同一轮 tool calls 可并发执行，但 think/reflection 应单独执行，形成决策停顿。
22. 为并发数、supervisor iteration、researcher tool calls 分别设置硬上限。
23. 搜索结果先 URL 去重，再做网页级摘要；摘要失败时可以降级返回 raw content。
24. Researcher 的最终输出应是 evidence-preserving compression，而不是极短摘要。
25. 把 summarization/research/compression/final writing 模型拆开配置，支持成本优化。
26. 最终 writer 不再搜索，确保阶段职责清晰；若要补证据，应回到 Research phase。
27. 对 token limit 做分阶段兜底：research compression 与 final report 分别处理。
28. 把 Agent trace 和 eval 绑定，以 trajectory 数据定位“搜索不足、重复研究、停止过早”等问题。

22. 从零实现一个 Open Deep Research MVP

22.1 最小组件

- 一个支持 structured output + tool calling 的 chat model。
- 一个 search tool（先只做 Tavily/Serper/自有搜索之一）。
- 一个 Scope 节点：messages → research_brief。
- 一个 Supervisor loop：Think / Delegate / Complete。
- 一个 Researcher loop：Search / Think / Complete。
- 一个 Compression 节点：raw researcher context → clean evidence report。
- 一个 Final Writer：brief + all compressed findings → final report。
- 三个预算参数：max_parallel_units、max_supervisor_iters、max_tool_iters。

22.2 推荐伪代码

Main

```
brief = scope(messages)
findings = []
for supervisor_iter in budget:
    plan = supervisor(brief, findings)
    if plan.complete: break
    topics = plan.topics[:max_parallel]
    results = await gather(*(researcher(topic) for topic in topics))
    findings.extend(results)
report = writer(brief, findings)
```

22.3 Researcher 伪代码

```
history = [system_prompt, topic]
for step in range(max_tool_iters):
    response = llm_with_tools(history)
    if no_tool_call(response): break
    tool_results = await run_tools_in_parallel(response.tool_calls)
    history += [response] + tool_results
return compress(history)
```

22.4 先不要做的东西

- 不要一开始接 10 个 search providers。先把 research policy 跑通。
- 不要先做复杂 memory。先确保一次 research task 能稳定结束并引用正确。
- 不要让每个 subagent 直接写最终章节。先统一压缩为 evidence。
- 不要把 graph 画得极复杂。把动态研究行为留在 tool loop。

23. 改进建议与下一代架构

23.1 Evidence Store 取代“全部 findings 塞给 Writer”

当前 writer 仍把所有 findings 一次性放入 prompt，超长时只能截断。下一步可将 compressed evidence 写入结构化 evidence store：claim、source URL、topic、timestamp、confidence、excerpt。Writer 按报告 outline 检索需要的证据，而不是一次性消费全部。

23.2 Coverage-aware Supervisor

让 Supervisor 维护显式 coverage matrix：Research Brief 中的 requirements $\times$ evidence status。每次研究后更新 coverage，再基于 missing/weak cells 决定下一轮。这样 stop condition 不再只是“模型觉得够了”。

23.3 Citation Verifier

最终报告完成后，对每个带引用 claim 抓取或复用 source evidence，运行 entailment verifier。若引用不支持 claim，则回退到 writer 修正或继续研究。

23.4 Search Provider Registry + Capability Negotiation

将 SearchAPI enum 改为可注册 provider，并让 tool metadata 声明

capabilities: web/news/finance/fulltext/site-filter/date-filter/cost。Supervisor 或 Researcher 可根据 research topic 选择工具，而不是全局固定一个 provider。

23.5 结果缓存与长期 Research Memory

高质量 research report 很昂贵。可以缓存 source fetch、page summaries、compressed findings，并为用户/组织建立 research artifact store。未来相似任务先检索历史 evidence，再决定是否需要新搜索。

下一代方向：从“Agent + Context Window”进一步走向“Agent + Evidence Store + Runtime Policy + Verifier”。这会比单纯继续增加 context window 更稳健。

结论：Open Deep Research 最重要的不是“多 Agent”，而是边界设计

Open Deep Research 是一个非常适合学习 Agent Engineering 的项目，因为它把多个关键原则放在了一个相对小的源码体量里：任务规格化、动态委派、上下文隔离、并发工具执行、压缩、模型路由、终止预算、LangGraph state、MCP、引用保真和最终统一写作。

更重要的是，它保留了一条清楚的架构演化轨迹：先通过 workflow 给弱模型增加结构；模型能力提升后，再主动删除会限制它的结构。这种“结构是可撤销的工程假设”值得沉淀为长期 know-why。

最终可复用原则：把稳定生命周期放在 Graph；把动态策略放在 Agent Loop；把高噪声数据隔离在 Subagent；在边界处压缩；把证据来源贯穿全链路；只在真正可独立的任务上并行。

附录 A. 核心源码阅读地图

文件	优先阅读内容	你会学到什么
src/open_deep_research/deep_researcher.py	四个主节点、Supervisor/Researcher subgraph、compression、final report	完整控制流
src/open_deep_research/configuration.py	并发/迭代预算、四类模型、SearchAPI、MCPCConfig	可配置 runtime
src/open_deep_research/state.py	Agent/Supervisor/Researcher state、override_reducer	状态边界与 reducer
src/open_deep_research/utils.py	Tavily、网页摘要、think_tool、MCP、get_all_tools	Tool Plane 与上下文预处理
src/open_deep_research/prompts.py	lead_researcher_prompt、research_system_prompt、compression、final report	Research policy 的 prompt 层
src/legacy/graph.py	早期 workflow	固定结构的优点与上限
src/legacy/multi_agent.py	早期 multi-agent	并行写作的协调问题
tests/run_evaluate.py	DeepResearch Bench eval	如何把架构改动绑定到指标

建议源码阅读路径

- 29. 先只看 `deep\_researcher\_builder` 的 4 个节点，建立大图。
- 30. 再看 `supervisor()` / `supervisor\_tools()`，理解“研究管理 loop”。
- 31. 再看 `researcher()` / `researcher\_tools()`，理解“执行 loop”。
- 32. 然后看 `compress\_research()`，理解 context boundary。
- 33. 接着看 `utils.get\_all\_tools()` 和 `tavily\_search()`，理解 data plane。
- 34. 最后读 prompts，观察 hard limits / scaling rules 如何把工程策略注入模型。
- 35. 回头看 legacy，理解哪些结构已经被 current 删除。

附录 B. 主要资料与引用

以下来源是本报告的主要事实依据。源码类结论以 2026-08-13 main 分支为准；博客用于解释设计动机；Benchmark 仓库用于说明 2026 evaluator 迁移。

[S1] LangChain Open Deep Research GitHub repository - https://github.com/langchain-ai/open_deep_research 仓库状态、README、评测记录、Legacy 说明。

[S2] Current main - `deep_researcher.py` -

https://github.com/langchain-ai/open_deep_research/blob/main/src/open_deep_research/deep_researcher.py 主图、Supervisor/Researcher loops、compression、final report。

[S3] Current main - `configuration.py` -

https://github.com/langchain-ai/open_deep_research/blob/main/src/open_deep_research/configuration.py 模型、并发、迭代、SearchAPI 与 MCP 配置。

[S4] Current main - `state.py` - https://github.com/langchain-ai/open_deep_research/blob/main/src/open_deep_research/state.py State schema 与 reducer。

[S5] Current main - `utils.py` - https://github.com/langchain-ai/open_deep_research/blob/main/src/open_deep_research/utils.py Tavily、网页摘要、MCP、工具装配。

[S6] Current main - `prompts.py` -

https://github.com/langchain-ai/open_deep_research/blob/main/src/open_deep_research/prompts.py Supervisor/Researcher hard limits、compression 与 final writing 规则。

[S7] LangChain official blog: Open Deep Research - <https://www.langchain.com/blog/open-deep-research> Scope-Research-Write、multi-agent 取舍、context engineering 设计动机。

[S8] Lance Martin: Learning the Bitter Lesson - https://rlancemartin.github.io/2025/07/30/bitter_lesson/ 架构从 workflow → multi-agent → current 的演化解释。

[S9] DeepResearch Bench official repository - https://github.com/Ayanami0730/deep_research_bench 100 个 PhD-level tasks、2026 evaluator 迁移至 GPT-5.5。

[S10] DeepResearch Bench paper - <https://arxiv.org/abs/2506.11763> Benchmark 的 RACE/FACT 设计与任务构建。

— End of Report —